

Finite local arithmetic at every prime

Exact Bezout identities, prime-power roots and addressable avoidance domains

A finite kernel-checked criterion for all-prime local admissibility, fixed-divisor obstructions for every signed integer, exact singular root lifting and nonenumerating interval counts.

132

reusable and generated
declarations audited

64

independent polynomial census
cases

512

independent prime-square root
counts

Local power-free arithmetic on the pinned Lean 4.20 toolchain. No global density, generic JSON interpreter or arbitrary repeated-factor admissibility claim.

Source, corpus receipts and reproduction commands accompany the repository snapshot.

What changed

PerfectPower now computes and proves whether a nonconstant integer polynomial has a fixed prime-power divisor at any prime. Given ascending integer coefficients and an exponent k greater than zero, the producer constructs a finite packet. For an admissible packet, the emitted theorem proves the all-prime condition. For an obstructed packet, it proves that every original integer input, including negative inputs, produces a value that is not k -free. This is a local arithmetic result; existence, asymptotic density and complete integral-point classification are different questions.

The release preserves the concurrent elliptic bridge, repeated saturation, cosmology and singlet-mediation commits. It changes neither their proof premises nor their physics interpretations. It uses the existing Lean 4.20 toolchain and cached Mathlib instead of importing OpenAI's much larger Lean 4.34 library.

The mathematical reduction

Write f for an integer polynomial. The producer computes integer polynomials u and v and a positive integer R satisfying $u f + v f' = R$. Polynomial extended Euclid runs over the rationals; clearing denominators gives the integer identity. The identity is checked directly in Lean. Its constant need not equal the resultant and is often smaller. For x^4+1 the certificate is $4f - Xf' = 4$, while the resultant is 256.

If p does not divide R and p divides $f(a)$, then p cannot divide $f'(a)$: evaluating the identity would otherwise imply p divides R . Two roots modulo p^k with the same residue modulo p consequently coincide modulo p^k . The proof factors the difference $f(b)-f(a)$ through $b-a$, evaluates the divided difference at a as $f'(a)$, and uses coprimality to cancel it.

Map the roots modulo p^k into the roots of f over the field with p elements. The preceding uniqueness makes this map injective. The reduced polynomial is nonzero at a good prime; otherwise the derivative identity would again imply p divides R . Its number of roots is bounded by its degree. Thus $\rho(f, p^k)$ is at most $\deg(f)$ at every good prime.

It follows that checking primes dividing R and primes at most $\deg(f)$ suffices. Every omitted prime exceeds the degree, and p^k is at least p for positive k . Therefore $\rho(f, p^k)$ is strictly below p^k . The finite criterion does not need irreducibility or a density theorem. Squarefree reducible inputs work. Repeated rational factors currently lack the required nonzero derivative Bezout identity and are rejected by this all-prime producer.

The Lean exceptional set contains the prime factors of R and the full natural range through the degree; its criterion explicitly restricts members to primes. The Python packet lists only the primes actually checked. A checked product of prime factors establishes the exceptional factor set by the fundamental theorem of arithmetic. This avoids relying on execution of a factorization program inside the final proof.

Literal roots and original integer coordinates

The roots for each exceptional modulus are emitted as literal finite sets. Their equality with the complete native Horner enumeration is checked by Lean's kernel. The coefficient polynomial is proved equal to the original supplied polynomial. A reusable modular-evaluation theorem then proves that, for every signed integer x , divisibility of $f(x)$ by the modulus is equivalent to membership of the canonical residue in that literal set. The packet thus provides original-coordinate statements, rather than just a root count attached to a normalized surrogate.

If one of these sets contains all residues modulo p^k , the obstruction theorem proves not $\text{PowerFree } k (f.\text{eval } x)$ for every x . Here PowerFree means no prime k -th power divides the value. Zero fails this condition. Units satisfy it. At $k=1$ it expresses absence of any prime divisor; at $k=2$ it is squarefreeness. Absence of a fixed divisor does not say that each value is power-free.

The public producer returns an unchecked-source flag because generating a source file is not compiling it. The release summary records the matching source hash and successful compilation of all retained fixtures. It does not claim that arbitrary later API packets have already been compiled. Generic JSON decoding and the implementation of the producer remain separate formalization obligations.

Executable root lifting and wheels

Prime-power roots are constructed by lifting each root r modulo p^j to $r+t p^j$ for every digit t from zero through $p-1$. Every child is tested exactly. Singular roots may branch, survive or disappear; no simple-root assumption is imposed on this computation. For x^4+4 at $p=2$ the root sets are $\{0\}$ modulo 2, $\{0,2\}$ modulo 4, and empty modulo 8. The complete native equalities independently check the final sets in the emitted fixtures.

An avoidance wheel combines distinct selected prime powers into one periodic original-coordinate domain. Each retained residue avoids divisibility at those selected primes. The wheel deduplicates repeated prime inputs, preserves negative-coordinate semantics and returns an exact rational periodic density. Its interval count sums floor-quotient counts over the retained residue classes. The count does not enumerate the interval. The stored examples use the closed interval from -10^{30} to 10^{30} , of width $2 \cdot 10^{30} + 1$.

Survivors of a finite wheel need not be globally power-free. Its density is the density of the finite-prime avoidance domain, not an enclosure of an infinite Euler product. An empty wheel is a global obstruction because every integer lies in one of the excluded residue classes. General wheel assembly, root-lifting algorithm correctness and all generated interval counts are tested Python execution, not new Lean compiler theorems.

Public use

The module `perfectpower.power_free_local` exports `local_admissibility`, `root_lifts`, `power_free_wheel` and `native_certificate`. The JSON query service adds `power_free_local`, `power_free_wheel` and `native_power_free_certificate` operations. Coefficients are integers in ascending order. The exponent defaults to two.

```
from perfectpower.power_free_local import local_admissibility, power_free_wheel
packet = local_admissibility([1, 0, 0, 0, 1], exponent=2)
assert packet['locally_admissible']
wheel = power_free_wheel([4, 0, 0, 0, 1], primes=[2, 3, 5],
                        lo=-10**30, hi=10**30)
print(wheel['count'], wheel['density'])

{"op": "native_power_free_certificate", "args": {"coefficients": [4, 0, 0, 0, 4], "exponent": 2}}
```

The public limits are degree 32, at most 64 supplied coefficients, coefficient height at most 512 bits, exponent at most 64 and bounded factorization and lifting work. Intermediate Bezout coefficients have a 16,384-bit cap. A root modulus is capped at 65,536, with a default of 4,096; native proof emission requires at most 4,096. Wheel periods are capped at one million. Exhaustion raises `WorkLimit` and never changes an incomplete search into an admissibility claim. Constant inputs and repeated-factor all-prime queries are rejected explicitly; repeated factors remain supported by the finite root and wheel operations.

Validation and exact scope

The reusable module has eleven audited theorem declarations. Eleven original-coordinate fixtures emit another 121 declarations, including complete root sets, exact cardinalities, original polynomial identities, Bezout identities and global admissibility or obstruction conclusions. Nine fixtures are admissible and two are obstructed. They cover monic, nonmonic, signed, reducible, degree-eight, exponent-one and exponent-three inputs, as well as an empty exceptional-prime case. All 132 declarations are checked for standard axioms only, without `sorryAx` or `Lean.ofReduceBool`.

The independent census covers 64 polynomials $x^d + a$ with degrees one through eight and eight signed nonzero constants. It checks 512 prime-square root counts by direct modular enumeration, verifies the omitted-prime degree bound and compares signed-interval wheel counts with direct scans. Eleven focused unit tests exercise construction identities, singular lifts, content obstructions, repeated-factor rejection, huge interval counts, public service dispatch and explicit budget stops. Related bridge, query-space, semilinear and service regressions are retained separately with their actual counts in the release summary.

Run `make power-free-check` to regenerate fixtures, build the module, compile both audits and run the focused tests. The check binds the complete generated source to its SHA-256 and verifies every printed axiom set. The complete historical repository build is not part of this focused check.

Provenance and next avenues

The simple-root uniqueness and root-count arguments adapt OpenAI math's `lean/OAI/NumberTheory/PowerFree/Sieve.lean` at commit `adc7f1241b42e322a6451854ab7e4b4c146bf78a`. The Apache-2.0 license and adaptation notice are retained in `third_party/openai_math`. The adaptation replaces the newer resultant API with an exact integer Bezout certificate and adds finite admissibility, native coefficient packets and signed-coordinate obstruction transport.

This gives the residue compiler and valuation engine a reusable all-prime local diagnostic. It also supplies a compact arithmetic boundary for future power-free density work: the local premise is now independently dischargeable. Proving a global density requires its analytic and geometric tail estimates; sampled local factors do not supply them. Other independent fronts remain repeated-factor all-prime diagnostics, generic JSON decoding, efficient Sturm variation, formal wheel construction, general Baker bounds and number-ring integral-point solvers. No new industrial speedup or physical prediction is claimed by this release.